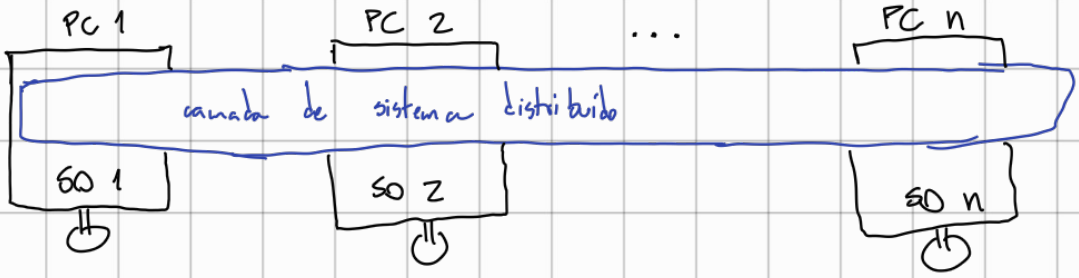
# Aula 2 - Sistemas Operacionais e SD

## conceitos de SO

## classificação de SO (tipos)

- monotarefa
- multitarefa
  - ↳ batch
  - ↳ tempo compartilhado
  - ↳ tempo real
- múltiplas CPU
  - ↳ paralelismo
  - ↳ computação paralela para SDs

## sistema distribuído



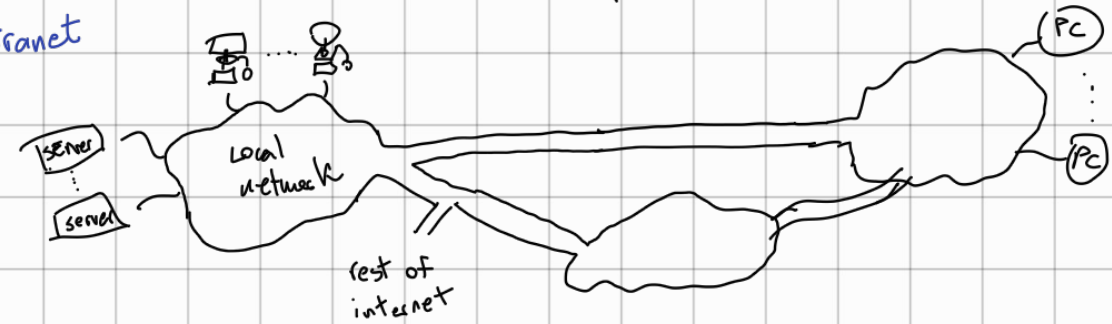

## problema dos relógios

- sincronização de relógios
- facilita a construção de sistemas distribuídos
- solução para exclusão mútua

## desafios

- segurança
- integridade
- heterogeneidade
- disponibilidade
- identidade do usuário
- escalabilidade
  - ↳ dividir para conquistar

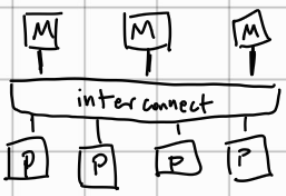
## exemplo de SD - intranet



AULA SD - 19/08/2026

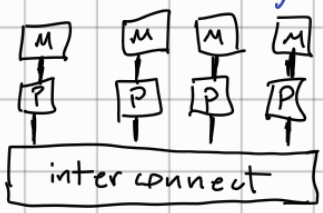
início → computação paralela
↓
computação distribuída

shared memory




↓
Facilidade sem escalabilidade

private memory

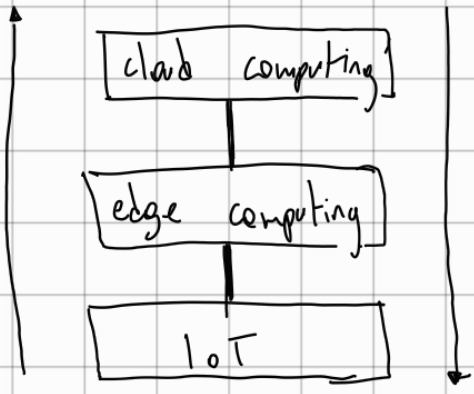



↓
escalabilidade porém exige comunicação

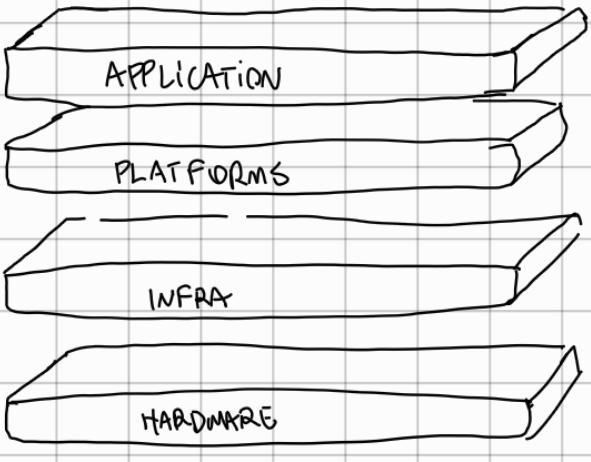
cluster
↳ computadores similares, mesmo SO e com conexão por rede veloz

grid
↳ sistema descentralizado, cada um em um domínio administrativo diferente, podendo ter diferentes hardware e software

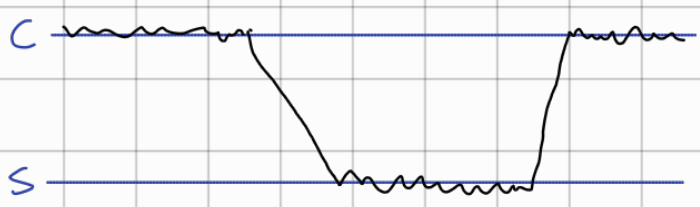
prós e contras




arquiteturas híbridas



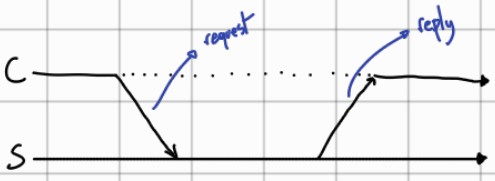

cliente - servidor
C S



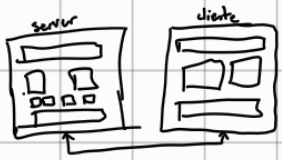
# AULA 26/05 - SISTEMAS DISTRIBUÍDOS

## arquitetura em camadas

modelo cliente - servidor

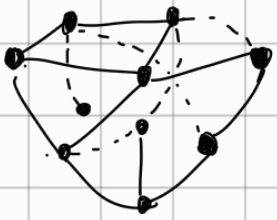



ex.: NFS




peer - to - peer

"pares em pares"



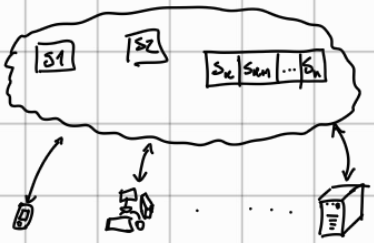
→ todos processos desempenham sendhantemente

→ resistência a falhas

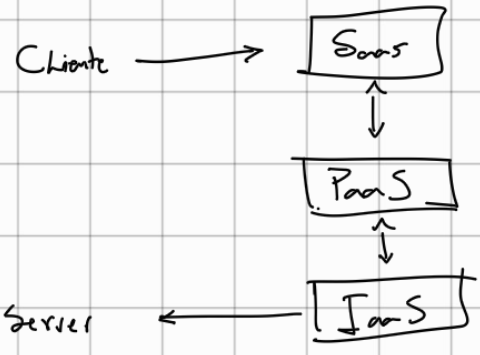
## arquiteturas híbridas

computação em nuvem




SaaS e PaaS (platform as a service) e IaaS (infra as a service)




## serverless




- serverless
- server contínuo
- não utilizado

tarefa computação sem servidor

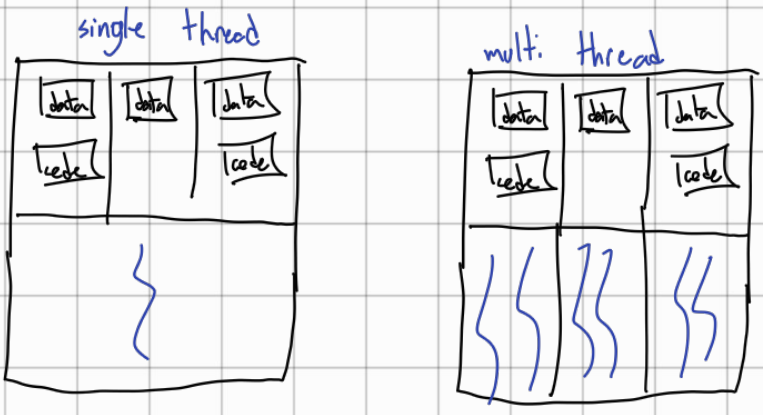
# AULA 02/09 - Sistemas Distribuídos PPGCC

## processo

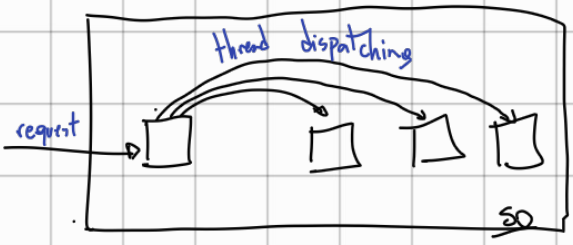
- é um "programa em execução"
- troca de processos ocorre por: salva registradores → mantém o que precisar em disco → troca → executa → interrompe

## threads

- são linhas de execução
- 1 processo : N threads



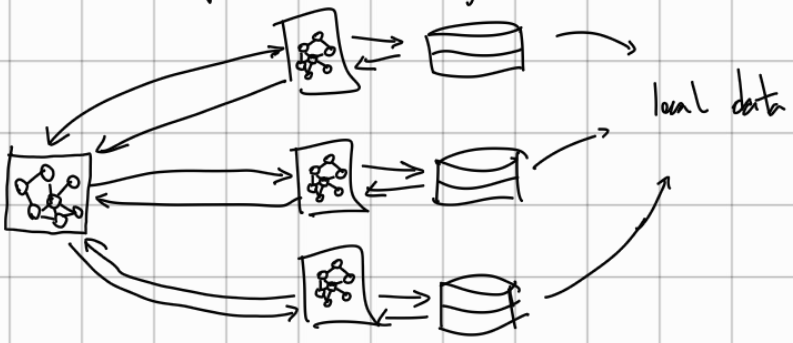

## servidor multi-threaded




## migração de código

- balanceamento de carga para desempenho
- pode usar diferentes métricas de decisão (fila CPU, utilização CPU...)
- precisa considerar custos: swap e comunicação

federated learning



# AULA 09/09 - SISTEMAS DISTRIBUÍDOS

partindo do conceito de processos cliente servidor

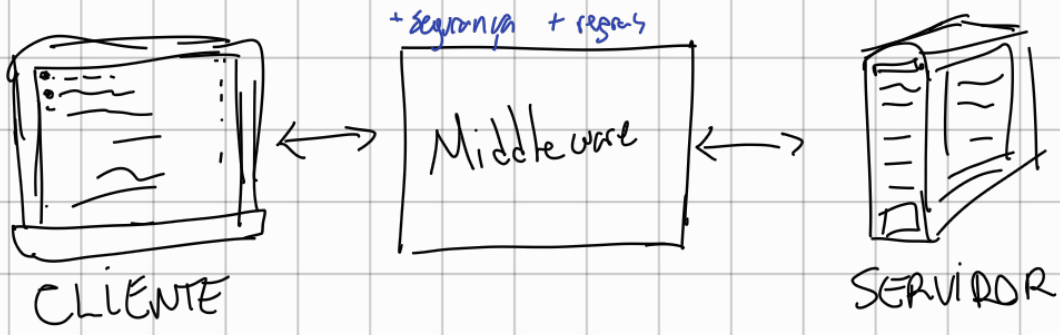
## cliente: sw para transparência

- cliente não deve saber o que ocorre por trás "isso não é problema meu"
- software do cliente mascara a ocorrência de falha (reconnect, cache ....)

## servidores: organização geral

- iterativo: um pedido por vez, é mais simples, porém um cliente demorado atrasa todos
- concorrente: (recebe cliente → passa para ajudante → livre pro próximo), clientes não ficam esperando
- servidores com estado: guarda o histórico ou dados da sessão do cliente (cache), server cai ↳ dados apagados
- servidores stateless: trata cada requisição como se fosse a primeira, implica em envios repetidos de dados

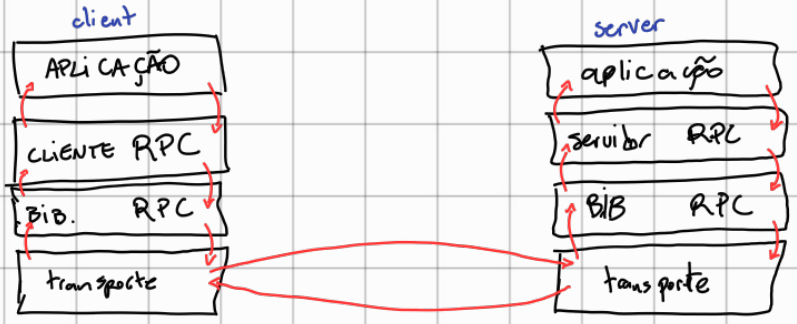
## middleware




## tipo de comunicação

- síncrono
  - ↳ assemelha-se ao iterativo, espera-se a confirmação do recebimento da mensagem (TCP)
- assíncrono
  - ↳ mais rápido, sem confirmação de recebimento (UDP)

## Remote Procedure Call



# Aula 23/09 - Sistemas Distribuídos

## Relógio Lógico

→ não é o foco em todos dispositivos terem o mesmo exato tempo, mas sim que respeite a ordenação causal

## Algoritmo de Lamport

→ a e b são eventos no mesmo processo ($a \rightarrow b$)

→ se a é um envio de mensagem, b é o evento de recebimento de mensagem

→ se dois eventos x e y acontecem em DIFERENTES processos, então não podemos afirmar algo sobre a ordem de execução, então, são CONCORRENTES

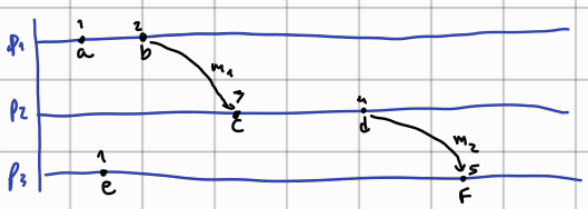



FIG. 1

→ cada processo $p_i$ tem seu próprio relógio $R_i$

→ pela FIG 1, afirmamos: $a \rightarrow b$, $b \rightarrow c$

→ os valores $R_i$ estão acima de cada $p_i$

$p_1$ envia $t=2$ em $m_1$, então $r_2 = \max(R_2, 2) + 1 = 3$

o tempo = qtd de eventos precedentes

# Aula 30/09 - Sistemas Distribuídos

## Tempo Lógico

O que importa são eventos e não a passagem do tempo real.
Cada "tic" é um evento.

## Lamport e a definição de Happened Before

Se um evento e aconteceu antes que e', então e potencialmente causou e', e também podemos dizer que e precede e' em uma ordem causal

$a \to b \iff$ a e b são do mesmo processo e a foi executado antes de b
ou
a é o envio de mensagem e b a sua recepção
ou
se existe transitividade $a \to c$ e $c \to b$

## Relógios lógicos → Relógio de Lamport

Seja e um evento e $C(e)$ o valor do relógio lógico quando associado a e, se $e \to e'$ então $C(e) < C(e')$

para definir a função C, seja $c_p$ um contador em p iniciado em 0,
$C(e) = ++ c_p$ quando e ocorre

cada processo tem seu relógio próprio

seja $c_p$ um contador em p com valor inicialmente igual a 0
se o evento é uma operação local
$c_p \leftarrow c_p + 1$
$C(e) \leftarrow c_p$

se e é o envio de mensagem
$c_p \leftarrow c_p + 1$
$C(e) \leftarrow c_p$
$C(e)$ é enviado com a mensagem como seu timestamp $t_s$

se e é uma recepção de uma mensagem com $t_s$
$c_p \leftarrow \max(c_p, t_s) + 1$
$C(e) \leftarrow c_p$